Lab 09
Memory, Cache, Loops and Array

Memory Instructions: In RISC-V assembly, memory instructions like lw (load word) and sw (store word) are essential for moving data between the processor's registers and main memory. The lw instruction loads a 32-bit value from a specified memory address into a register, allowing the CPU to perform computations on data stored in memory. Conversely, sw stores a value from a register back into memory, preserving the result for later use or for other parts of the program.

Why:

- CPU registers are **limited in number and size**, so most data must reside in memory.
- lw and sw allow programs to **access larger datasets** beyond registers.
- They enable **updating variables** and **maintaining results** across program execution.
- Facilitate **efficient computation** by moving data between memory and registers.
- Demonstrate a typical **load-compute-store cycle**:
  1. Load values from memory (lw)
  2. Compute using registers (add)
  3. Store results back to memory (sw)

```
# memory[0] = ?, memory[4] = ?
lw t0, 0(x0)
lw t1, 4(x0)
add t2, t0, t1
sw t2, 8(x0) # store result at memory[8]
```

Explanation:

- RISC-V memory access uses base + offset addressing: lw t0, OFFSET(BASE_REGISTER).
- 0(x0) → base = x0 (always 0), offset = 0 → memory address = 0 → loads 1st word.
- 4(x0) → base = x0 (0), offset = 4 → memory address = 4 → loads 2nd word.

- Each .word in memory occupies 4 bytes, so addresses increment by 4 for each word: 0, 4, 8, 12…
- Example mapping:
    - memory[0] = 5 → first word
    - memory[4] = 7 → second word
    - memory[8] = result → third word
- x0 acts as a zero base, offsets point to word positions in memory.

Output

Name	Alias	Value
x0	zero	0x00000000
x1	ra	0x00000000
x2	sp	0x7ffffff0
x3	gp	0x10000000
x4	tp	0x00000000
x5	t0	0x00002283
x6	t1	0x00402303
x7	t2	0x00404586

Output in memory

Memory viewer

Address	Word	Byte 0	Byte 1	Byte 2	Byte 3
0x00000028	X	X	X	X	X
0x00000024	X	X	X	X	X
0x00000020	X	X	X	X	X
0x0000001c	X	X	X	X	X
0x00000018	X	X	X	X	X
0x00000014	X	X	X	X	X
0x00000010	0x00000000	0x00	0x00	0x00	0x00
0x0000000c	0x00702423	0x23	0x24	0x70	0x00
0x00000008	0x00404586	0x86	0x45	0x40	0x00
0x00000004	0x00402303	0x03	0x23	0x40	0x00
0x00000000	0x00002283	0x83	0x22	0x00	0x00

Explanation:

- **lw t0, 0(x0)**: Loads the 32-bit value located at **memory address 0** into register **t0**.
- **lw t1, 4(x0)**: Loads the 32-bit value located at **memory address 4** into register **t1**.
- **add t2, t0, t1**: Adds the two register values (from t0 and t1) and writes the sum into **t2**.
- **sw t2, 8(x0)**: Stores the computed value in **t2** into **memory address 8**.

Memory instructions with actual data.

```
.data
num1: .word 10 # memory[0] = 10
num2: .word 20 # memory[4] = 20
result: .word 0 # memory[8] = 0 (to store result)

.text
la t0, num1 # load address of num1
lw t1, 0(t0) # load value 10 into t1

la t0, num2 # load address of num2
lw t2, 0(t0) # load value 20 into t2

add t3, t1, t2 # t3 = 10 + 20

la t0, result # load address of result
sw t3, 0(t0) # store sum 30
```

Explanation:

- .data section → declares and initializes memory variables (num1 = 10, num2 = 20, result = 0).
- .text section → contains the instructions to execute.
- la t0, num1 → loads the **address** of num1 into register t0.
- lw t1, 0(t0) → loads the **value** at num1 (10) into t1.
- la t0, num2 → loads the **address** of num2 into t0.
- lw t2, 0(t0) → loads the **value** at num2 (20) into t2.
- add t3, t1, t2 → adds the two values (10 + 20) and stores the result in t3.

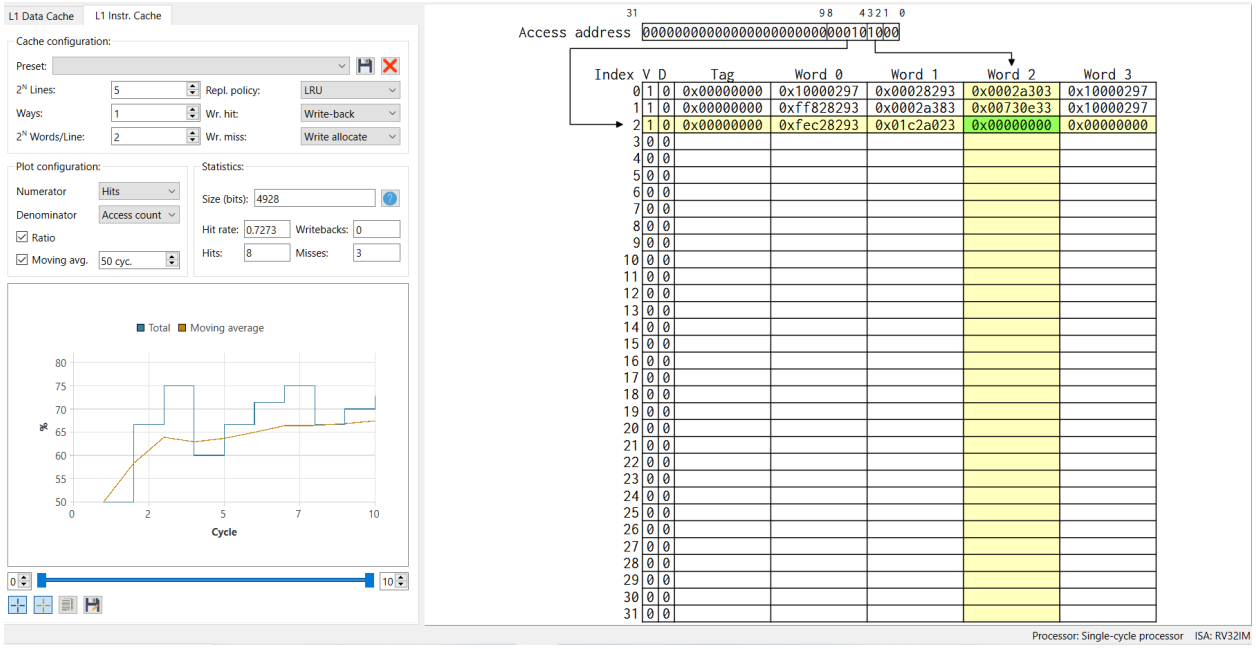
- la t0, result → loads the **address** of result into t0.
- sw t3, 0(t0) → stores the sum (30) into memory at result.
- Overall → demonstrates a **load-compute-store cycle** using initialized memory values.

Output in memory

Memory viewer

Address	Word	Byte 0	Byte 1	Byte 2	Byte 3
0x10000028	X	X	X	X	X
0x10000024	X	X	X	X	X
0x10000020	X	X	X	X	X
0x1000001c	0x00000000	0x00	0x00	0x00	0x00
0x10000018	X	X	X	X	X
0x10000014	X	X	X	X	X
0x10000010	X	X	X	X	X
0x1000000c	0x00000000	0x00	0x00	0x00	0x00
0x10000008	0x0000001e	0x1e	0x00	0x00	0x00
0x10000004	0x00000014	0x14	0x00	0x00	0x00
0x10000000	0x0000000a	0x0a	0x00	0x00	0x00

Cache View (Instruction)



Cache table explanation (right side of the image)

- Index → cache line index (0–31)
- V → valid bit (1 = data present)
- D → dirty bit (1 = modified, not written back yet)
- Tag → upper address bits identifying memory block
- Word 0–3 → data in cache line (each line has 4 words)
- Line 0: contains previous data (`0x10000297`, `0x00028293`, …)
- Line 1: contains another block of memory
- Line 2: shows your recently stored value (`0x00000000` in Word 2 → likely corresponds to `result`)

Address mapping

- Access address: 0000...0100 = decimal 4 → corresponds to memory[4] (num2)
- Cache **index calculation**: lower bits of address select line → you see index 2 highlighted
- Tag: upper bits of address → helps identify which memory block is in the cache line

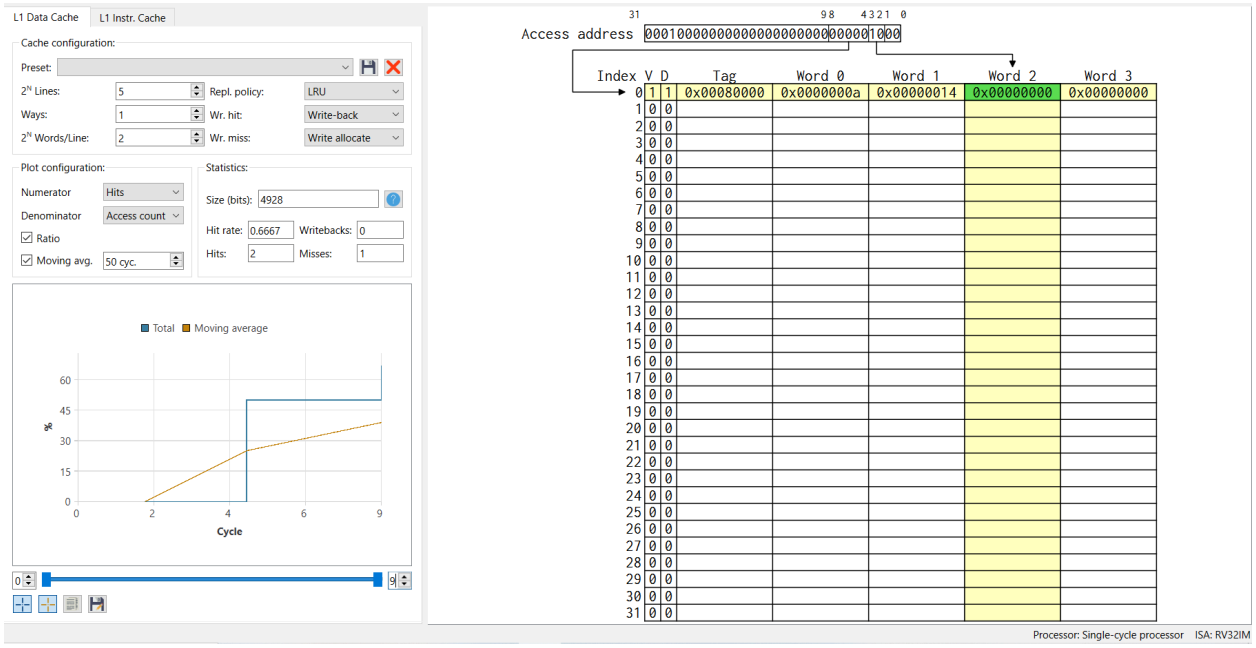
Statistics & plot (bottom-left)

- Hit rate: 0.7273 → ~73% of accesses found in cache
- Hits = 8, Misses = 3 → initial loads (num1, num2, result) cause misses; subsequent accesses hit cache
- Moving average shows how the hit rate evolves over cycles

Summary

- This program accesses memory 3 times for num1, num2, and result.
- Each lw/sw triggers a cache lookup, which may cause a miss or hit.
- Cache table shows how data is stored in L1, which line, which word, and if it's dirty/valid.
- The graph tracks overall cache performance as the program runs.

Cache View (Data)



Arrays

```
.data
myArray: .word 5, 10, 15   # Define an array with 3 numbers

.text
   la   t3, myArray      # Load base address of array into t3

   lw   t0, 0(t3)        # Load first element (5) into t0
   lw   t1, 4(t3)        # Load second element (10) into t1
   lw   t2, 8(t3)        # Load third element (15) into t2
```

Explanation:

- .data section declares myArray with 3 elements.
- la t3, myArray loads the base address of the array.
- Each lw instruction loads one element into a temporary register.
- Offsets increase by 4 bytes per word: 0, 4, 8.

Output

Name	Alias	Value
x0	zero	0x00000000
x1	ra	0x00000000
x2	sp	0x7ffffff0
x3	gp	0x10000000
x4	tp	0x00000000
x5	t0	0x00000005
x6	t1	0x0000000a
x7	t2	0x0000000f

Data in memory

Memory viewer

Address	Word	Byte 0	Byte 1	Byte 2	Byte 3
0x10000028	X	X	X	X	X
0x10000024	X	X	X	X	X
0x10000020	X	X	X	X	X
0x1000001c	X	X	X	X	X
0x10000018	X	X	X	X	X
0x10000014	X	X	X	X	X
0x10000010	X	X	X	X	X
0x1000000c	X	X	X	X	X
0x10000008	0x0000000f	0x0f	0x00	0x00	0x00
0x10000004	0x0000000a	0x0a	0x00	0x00	0x00
0x10000000	0x00000005	0x05	0x00	0x00	0x00

Loop example

```
addi t0, x0, 5 # t0 = 5 (loop counter)

LOOP:
   addi t0, t0, -1 # t0 = t0 - 1
   bne t0, x0, LOOP # If t0 != 0, repeat loop
```

Loop using two registers (Like in EMU8086)

```
addi t0, x0, 0      # counter = 0
addi t1, x0, 5      # limit = 5
```

```
LOOP_TWO:
   addi t0, t0, 1    # counter++
   bne  t0, t1, LOOP_TWO  # stop at 5
```

Loop with condition

```
addi t0, x0, 5       # counter = 5
addi t1, x0, 0       # sum = 0

LOOP_COND:
   addi t2, x0, 3    # value to skip = 3
   beq  t0, t2, SKIP # if counter == 3, skip

   addi t1, t1, 1    # sum++

SKIP:
   addi t0, t0, -1   # counter--
   bne  t0, x0, LOOP_COND
```

Exercise:

Task 1: Define array, iterate using loop and computer sum of array.

Task 2: Explore stack operations in Ripes with an example code.

Task 3: Pop and push 5 values, and analyze the cache (both instruction and data).